# LABORATORY PROJECT REPORT

# SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

# EXPERIMENT 3A

## Section 1

## Group 2

## Semester 2 2025/2026

| NO | NAME | MATRIC NO |
|---|---|---|
| 1 | AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI | 2310663 |
| 2 | ALYA SYAFINA BINTI ABDUL RAZAK | 2218084 |

DATE OF SUBMISSION: 25 MARCH 2026

## Abstract

This study examines the implementation of real-time serial communication between an Arduino Uno microcontroller and a Python-based computing environment. The primary objective was to establish a seamless data pipeline, beginning with a potentiometer to generate variable analog signals which were then processed by the microcontroller and transmitted via a USB serial link. By utilizing the *pyserial* library, the Python script was able to move beyond static data logging to achieve a dynamic, graphical representation of the sensor's behavior.

The core of the experiment focused on the technical nuances of system integration, particularly the precision required for analog-to-digital conversion and the necessity of exact baud rate synchronization to maintain signal integrity. By managing these hardware inputs within a high-level software environment, the setup demonstrated how raw electrical signals can be effectively transformed into actionable visual data. Ultimately, the results confirm that robust serial exchange remains a fundamental requirement for the synchronization of modern mechatronic and embedded systems.

## Table of Contents

1.0 **Introduction**

2.0 **Materials and Equipment**

2.1 Electronic Components

2.2 Equipment and Tools

3.0 **Experimental Setup**

4.0 **Methodology**

5.0 **Data Collection**

6.0 **Data Analysis**

7.0 **Results**

8.0 **Discussion**

9.0 **Conclusion**

10.0 **Recommendations**

11.0 **References**

## 1.0 Introduction

Modern mechatronics relies heavily on the seamless exchange of information between microcontrollers and high-level computing environments. This interaction is foundational for data acquisition, system monitoring, and precise control. This experiment explores these principles by establishing a functional link between an Arduino board and a Python-based interface, specifically to monitor and visualize inputs from a potentiometer.

In this experiment, the Arduino serves as the primary acquisition unit, transforming the analog voltage from the sensor into digital values. These signals are then streamed via a USB serial connection to a computer, where a Python script that uses the *pyserial* library intercepts and displays the readings dynamically. This setup provides a hands-on look at critical technical hurdles, such as ensuring signal integrity through analog-to-digital conversion and maintaining strict synchronization by baud rate alignment.

Beyond the immediate hardware configuration, the experiment bridges the gap between low-level firmware and high-level software. It reinforces the programming logic required to handle real-time data streams while demonstrating the core principles of sensor interfacing and visualization. Ultimately, mastering these communication parameters is essential for developing the automated, responsive systems that define current mechatronic applications.

## 2.0 Materials and Equipment

### 2.1 Electronic Components

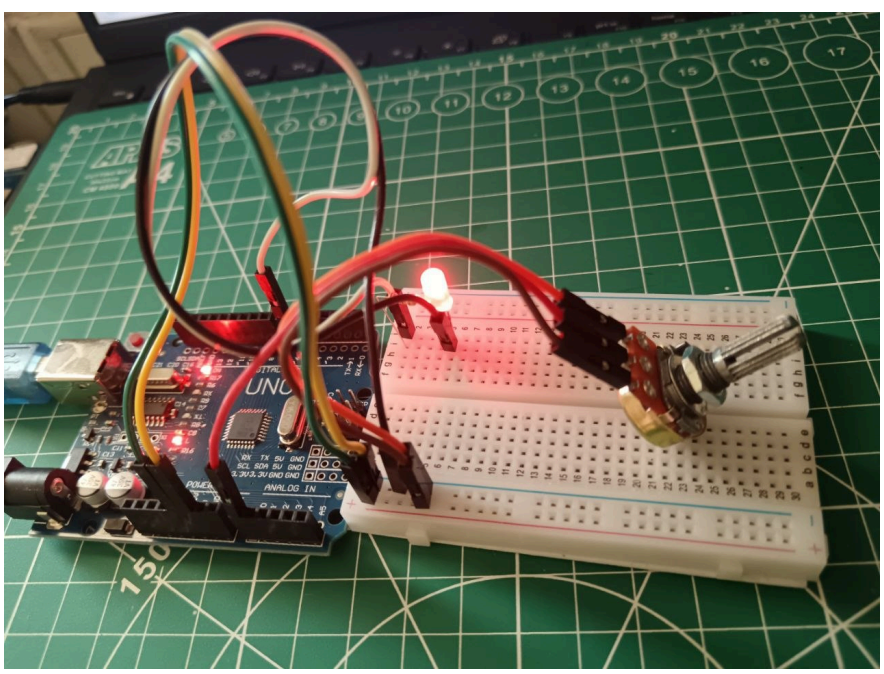
- Arduino Uno – Microcontroller board for controlling the display
- Potentiometer – A variable resistor used to generate varying analog voltage levels.
- 220-Ohm Resistors – Current-limiting resistors for each segment of the display
- LED – Optional visual indicator for circuit verification or output indication.
- Jumper Wires – For making connections between components
- Breadboard – For assembling the circuit without soldering

2.2 Equipment and Tools

- Arduino IDE – Software for writing and uploading code to the Arduino Uno
- USB Cable – For connecting the Arduino Uno to a computer
- Power Supply (5V via USB or external source) – To power the circuit
- Python with PySerial Library – Used to establish and manage serial communication with the Arduino and display real-time data.

## 3.0 <u>Experimental Setup</u>

1. The potentiometer was connected to the Arduino Uno with one leg to 5V pin on arduino, the other leg to gnd, and middle leg to pin A0.
2. Arduino was connected to the computer via USB, as shown in Figure 1, and run the Python script to begin reading potentiometer values in the terminal.
3. The potentiometer knob was adjusted to observe the values displayed in real-time
4. Serial plotter in Arduino IDE was opened to view the real time data fron the potentiometer adjustment.
5. An LED connected to the pin 9 and 220-ohm resistor added as an optional output indicator connected to a digital pin to observe visual feedback after the potentiometer value exceeded half.
6. All components were connected using jumper wires on a breadboard for a secure and solderless setup.



## 4.0 Methodology

1. Setup the Arduino Uno
2. IDE code implementation
3. Testing the terminal in python
4. Use the serial plotter in IDE
5. Code snippet

### 4.1 Detail steps

1. Assemble the circuit as stated in the experimental setup.
2. Connect the arduino to the computer via usb cable, and upload the code to arduino IDE.
3. Open the serial plotter on the Arduino to see the real time values of the potentiometer.

### 4.2 Programming codes

**Arduino code**

```
const int potPin = A0;    // Potentiometer connected to analog pin A0

const int ledPin = 9;     // LED connected to digital pin 9 (through 220Ω resistor)


void setup() {

  Serial.begin(9600);

  pinMode(ledPin, OUTPUT);  // Set LED pin as output

}


void loop() {

  int potValue = analogRead(potPin); // Read potentiometer value (0–1023)
```

```
  Serial.println(potValue);


  // Check if potentiometer value exceeds half of maximum (1023/2 = 511)
  if (potValue > 511) {
    digitalWrite(ledPin, HIGH); // Turn LED ON
  } else {
    digitalWrite(ledPin, LOW);  // Turn LED OFF
  }
  delay(1000); // Wait for 1 second
}
```

**Python code**

```
import serial
ser = serial.Serial('COM5', 9600)
try:
    while True:
        pot_value = ser.readline().decode().strip()
        print("Potentiometer Value:", pot_value)
except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
```

## 5.0 <u>Data Collection</u>

| Value | LED |
|---|---|
| 0 | OFF |
| 511 | OFF |
| 512 | ON |

## 6.0 Data Analysis

The Arduino circuit reads an analog input from the potentiometer and uses it to control an LED. As the potentiometer is adjusted, it generates a voltage ranging from 0 V to 5 V. The Arduino converts this voltage into a digital value between 0 and 1023. The program then checks this value against a threshold of 511. Initially, the value is 0, so the LED stays off. Once the reading exceeds 511, the LED connected to pin 9 turns on.

The Python program supports this system by continuously receiving and displaying the potentiometer values sent from the Arduino through the serial port. This enables real-time monitoring of how changes in the potentiometer position influence both the analog readings and the LED's behavior.

## 7.0 Results

The system functioned smoothly throughout the experiment, with potentiometer readings accurately transmitted from the Arduino to Python in real time. As the potentiometer was adjusted, the LED switched on once the value exceeded 511. No delays or errors were observed, and the system responded quickly to changes in the potentiometer's position, indicating that both the hardware connections and Arduino program were operating correctly. The consistent performance also demonstrated reliable serial communication and proper integration between the Arduino and Python systems.

## 8.0 Discussion

In this project, the Arduino board was connected to a potentiometer to measure analog voltage values that change as the knob is turned. Both Python and the Arduino Serial Plotter were used to visualize the data transmitted from the potentiometer to the computer through a serial connection. The experiment successfully demonstrated real-time acquisition, transmission, and visualization of analog sensor data.

The serial readings showed proportional voltage changes as the potentiometer was adjusted, highlighting the linear relationship between its position and the analog-to-digital converter (ADC) output values, which range from 0 to 1023 for a 10-bit ADC. The use of a Python script also provided flexibility for data processing, logging, and potential integration into more advanced control systems.

Additionally, the LED served as a visual indicator of the potentiometer's position when included in the circuit. It turned on when the potentiometer reading exceeded half of its range, confirming that both the serial communication and program logic were functioning correctly.

Some challenges encountered included avoiding conflicts between the Arduino Serial Plotter and Python serial communication, as well as ensuring that the COM port and baud rate settings were correctly configured. Proper synchronization of these parameters was essential for smooth and reliable data transfer.

## 9.0 Conclusion

In conclusion, the objective of the experiment is to use USB serial communication to transfer potentiometer readings from the Arduino to Python, which was successfully achieved. Real-time data transmission confirmed that the analog readings from the Arduino could be accurately recorded and analyzed in Python. The inclusion of the LED clearly demonstrated how

external components can be controlled based on the transmitted data. Overall, this setup highlights the flexibility of Arduino–Python integration for applications such as automation, monitoring, and data collection.

## 10.0 Recommendations

To avoid communication conflicts, it is recommended to close the Serial Plotter before running the Python script. Always ensure that the baud rate and COM port are correctly configured before starting the experiment. Implementing data smoothing in Python can help improve the stability of readings, while visualization tools such as Matplotlib can enhance data presentation. Additionally, the system can be expanded by integrating more sensors or actuators, and incorporating error handling is important to ensure reliable and consistent communication.

## 11.0 References


- Week 3: Serial Communication (Version 3)

  https://drive.google.com/drive/folders/1QecGGN96D76UL2gwBjevFwiEXv2UEniO

- Python for undergraduate engineers: Using python and arduino to read sensor

  https://pythonforundergradengineers.com/python-arduino-potentiometer.html


## 12. Acknowledgments


Special thanks to ZULKIFLI BIN ZAINAL ABIDIN for his sharing the tips and support during this experiment, even though it is a take-home experiment.

## Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

| | | |
|---|---|---|
| Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI | Read | [/] |
| Matric Number: 2310663 | Understand | [/] |
| Signature: SYAHMI | Agree | [/] |

| | | |
|---|---|---|
| Name: ALYA SYAFINA BINTI ABDUL RAZAK | Read | [/] |
| Matric Number: 2218084 | Understand | [/] |
| Signature: SYAF | Agree | [/] |